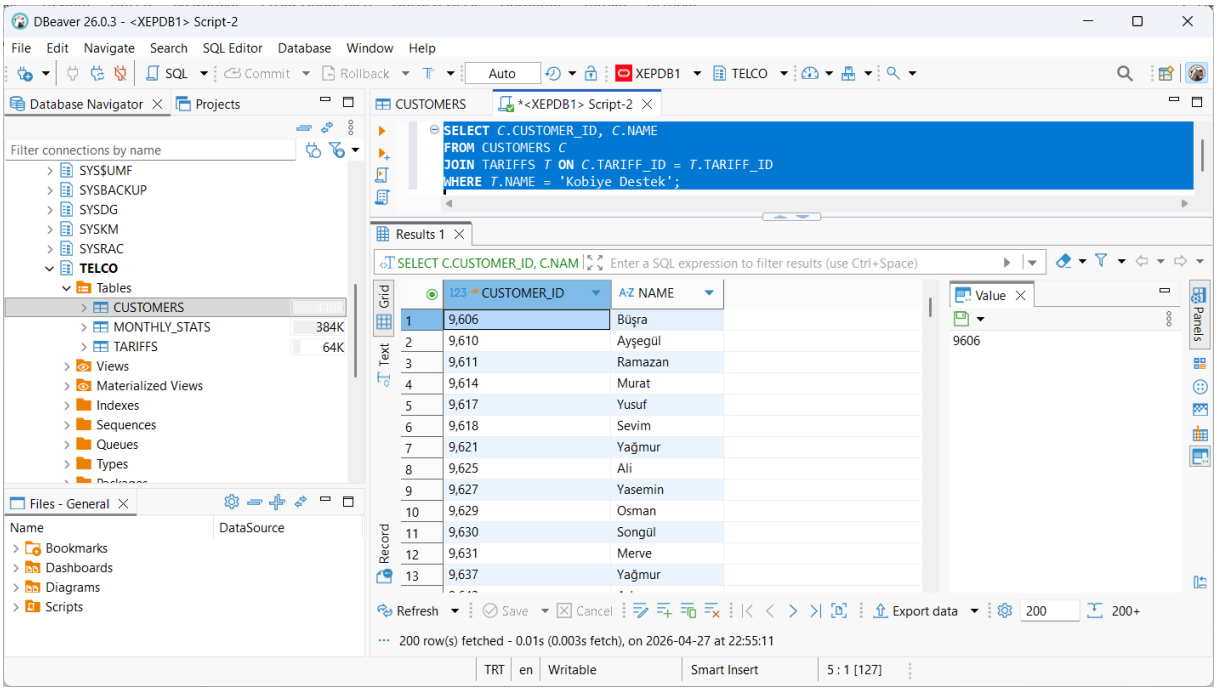
1.1 List the customers who are subscribed to the 'Kobiye Destek' tariff.

To find these specific customers, we bring together the CUSTOMERS and TARIFFS tables using a simple join on the tariff ID. Once they are connected, we just filter the results to only show the folks who are on the 'Kobiye Destek' plan. This gives us a clean list containing just the customer IDs and their names, making it easy to identify them.

Query:

```
SELECT C.CUSTOMER_ID, C.NAME
FROM CUSTOMERS C
JOIN TARIFFS T ON C.TARIFF_ID = T.TARIFF_ID
WHERE T.NAME = 'Kobiye Destek';
```

Output:



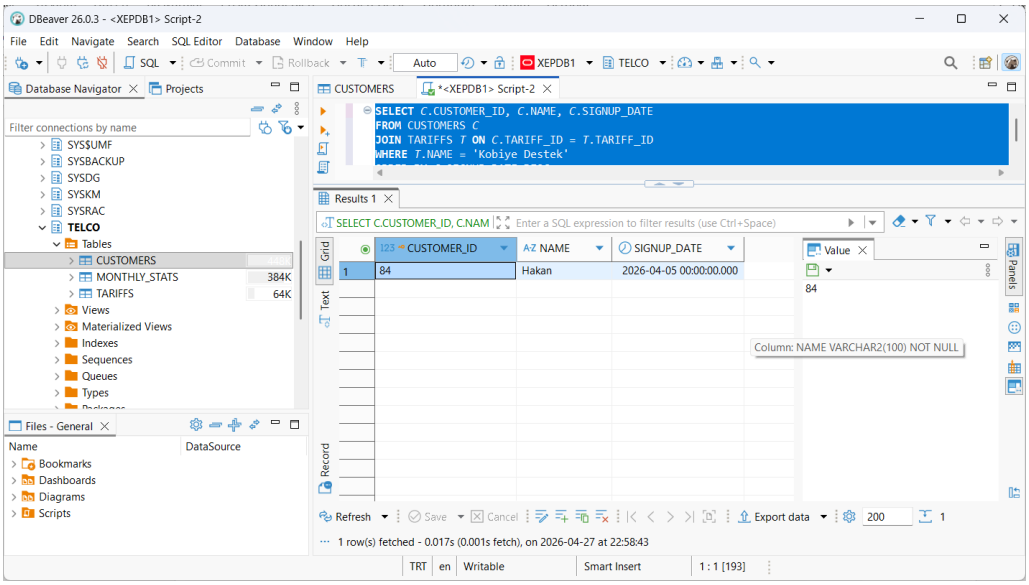
1.2 Find the newest customer who subscribed to this tariff ('Kobiye Destek').

Finding the newest subscriber requires us to first filter down to the 'Kobiye Destek' tariff, just like we did in the previous step. After that, we sort everyone based on their signup date, putting the most recent dates right at the top. By grabbing just the very first row from this sorted list, we can instantly pinpoint our newest customer.

Query:

```
SELECT C.CUSTOMER_ID, C.NAME, C.SIGNUP_DATE
FROM CUSTOMERS C
JOIN TARIFFS T ON C.TARIFF_ID = T.TARIFF_ID
WHERE T.NAME = 'Kobiye Destek'
ORDER BY C.SIGNUP_DATE DESC
FETCH FIRST 1 ROWS ONLY;
```

Output:



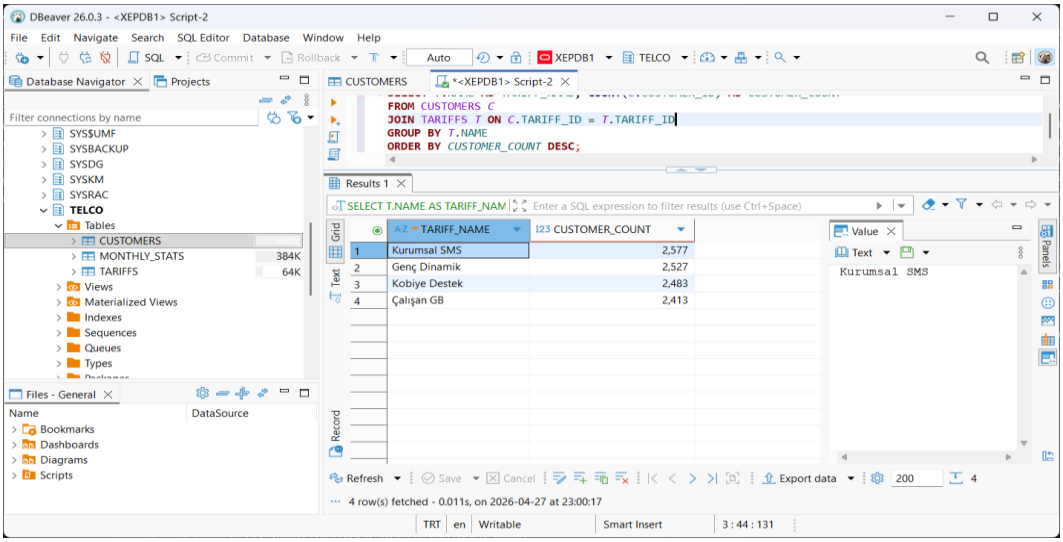
2.1 Find the distribution of tariffs among the customers.

We want to see how popular each tariff is, so we group our customers based on the tariff they've chosen. By joining with the TARIFFS table, we can display the actual, easy-to-read names of the tariffs rather than just a bunch of IDs. Finally, we count how many customers fall into each group and order them, showing the most popular plans at the top.

Query:

```
SELECT T.NAME AS TARIFF_NAME, COUNT(C.CUSTOMER_ID) AS CUSTOMER_COUNT
FROM CUSTOMERS C
JOIN TARIFFS T ON C.TARIFF_ID = T.TARIFF_ID
GROUP BY T.NAME
ORDER BY CUSTOMER_COUNT DESC;
```

Output:



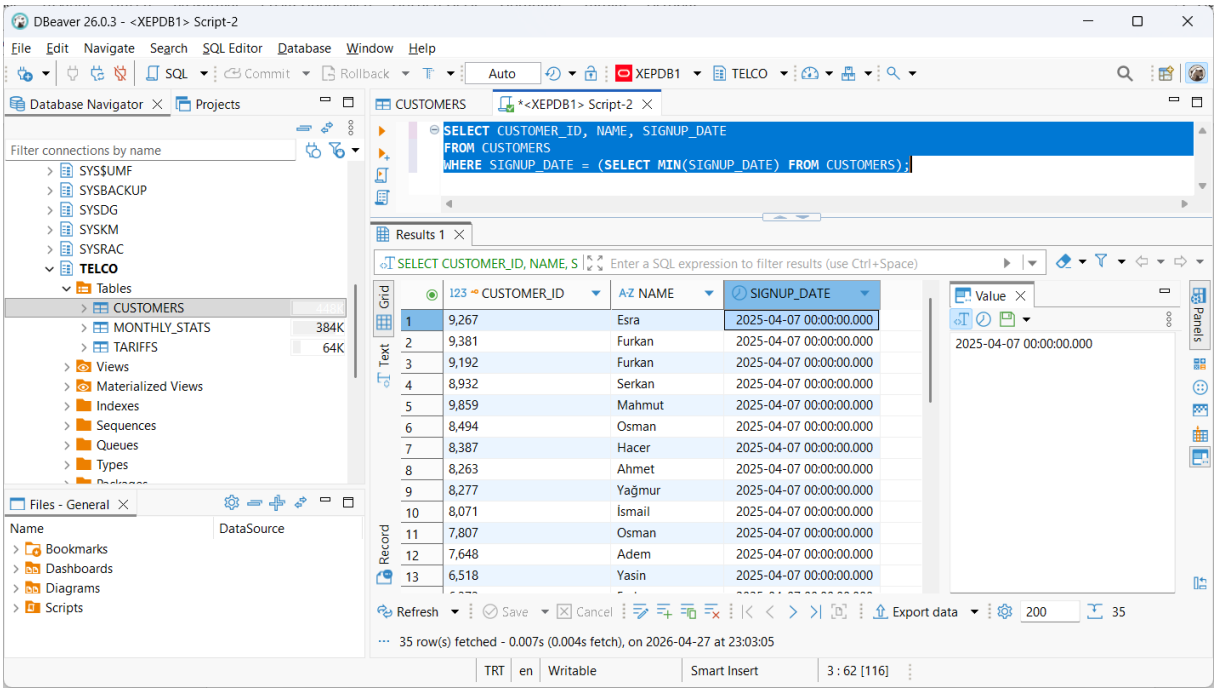
3.1 Identify the earliest customers to sign up.

Sometimes the first customers don't necessarily have the lowest ID numbers, so we have to look strictly at their signup dates. We do this by finding the absolute minimum signup date across the entire customer base using a subquery. Then, we pull up all the records that match this exact date, which safely handles cases where multiple people might have joined on that very first day.

Query:

```
SELECT CUSTOMER_ID, NAME, SIGNUP_DATE
FROM CUSTOMERS
WHERE SIGNUP_DATE = (SELECT MIN(SIGNUP_DATE) FROM CUSTOMERS);
```

Output:



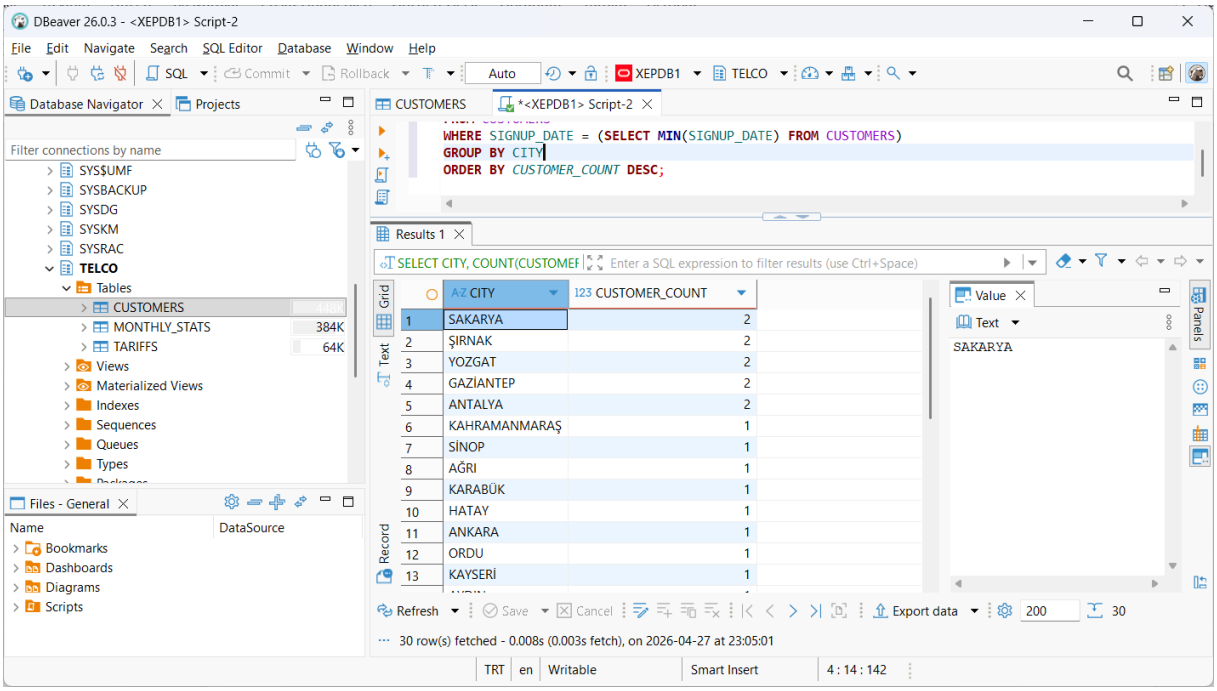
3.2 Find the distribution of these earliest customers across different cities.

Building on our previous search, we isolate that group of pioneer customers who joined on the very first day. We then organize them based on the cities they live in to see where our initial user base originated. Counting them up and sorting the results gives us a great geographical breakdown of our early adopters.

Query:

```
SELECT CITY, COUNT(CUSTOMER_ID) AS CUSTOMER_COUNT
FROM CUSTOMERS
WHERE SIGNUP_DATE = (SELECT MIN(SIGNUP_DATE) FROM CUSTOMERS)
GROUP BY CITY
ORDER BY CUSTOMER_COUNT DESC;
```

Output:



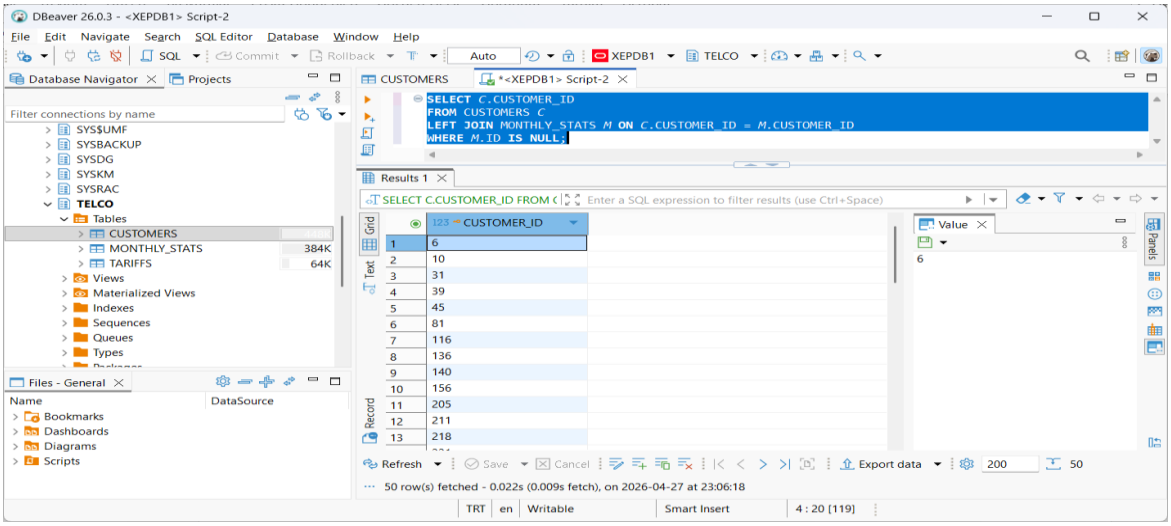
4.1 Identify the IDs of these missing customers (who don't have monthly records).

To spot the missing records, we take all our customers and try to match them with their monthly usage stats using a LEFT JOIN. If a customer exists but doesn't have a matching usage record, the database will fill that gap with a NULL value. By specifically searching for these NULL IDs, we can quickly isolate the customers affected by the insertion error.

Query:

```
SELECT C.CUSTOMER_ID
FROM CUSTOMERS C
LEFT JOIN MONTHLY_STATS M ON C.CUSTOMER_ID = M.CUSTOMER_ID
WHERE M.ID IS NULL;
```

Output:



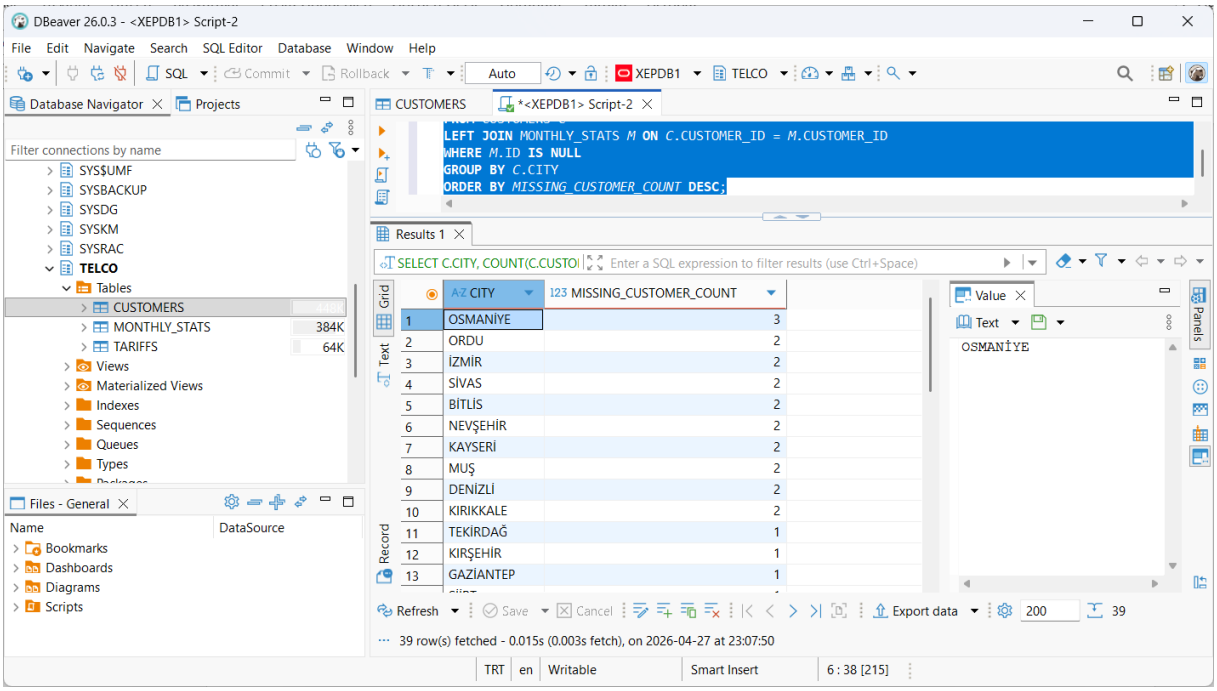
4.2 Find the distribution of these missing customers across different cities.

Once we know how to find the customers with missing data, we can take it a step further to see if the issue is localized. We use the same technique to find the missing records, but this time we group the affected users by their cities. This helps us understand the impact of the error across different regions by giving us a clear count for each city.

Query:

```
SELECT C.CITY, COUNT(C.CUSTOMER_ID) AS MISSING_CUSTOMER_COUNT
FROM CUSTOMERS C
LEFT JOIN MONTHLY_STATS M ON C.CUSTOMER_ID = M.CUSTOMER_ID
WHERE M.ID IS NULL
GROUP BY C.CITY
ORDER BY MISSING_CUSTOMER_COUNT DESC;
```

Output:



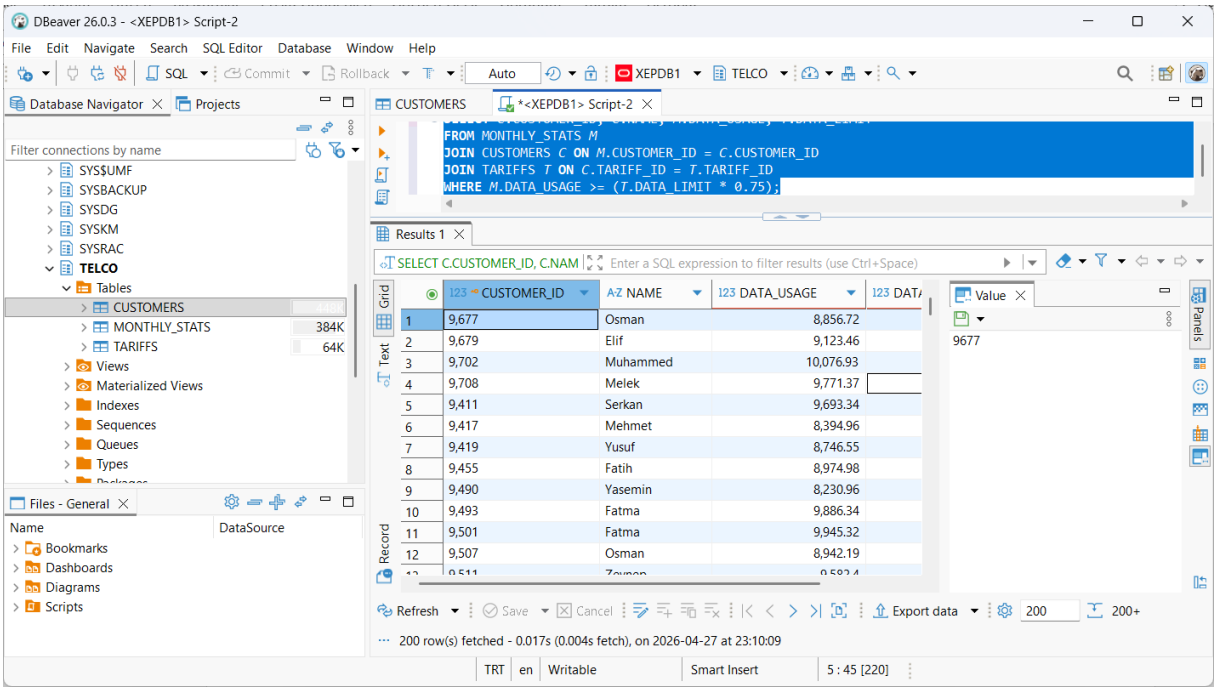
5.1 Find the customers who have used at least 75% of their data limit.

We need to compare what the customers actually used against what their tariff allows, which means bringing all three tables together. By multiplying the data limit by 0.75, we calculate the threshold for heavy usage. Anyone whose usage hits or exceeds this customized threshold gets pulled into our final list, helping us identify high-data consumers.

Query:

```
SELECT C.CUSTOMER_ID, C.NAME, M.DATA_USAGE, T.DATA_LIMIT
FROM MONTHLY_STATS M
JOIN CUSTOMERS C ON M.CUSTOMER_ID = C.CUSTOMER_ID
JOIN TARIFFS T ON C.TARIFF_ID = T.TARIFF_ID
WHERE M.DATA_USAGE >= (T.DATA_LIMIT * 0.75);
```

Output:



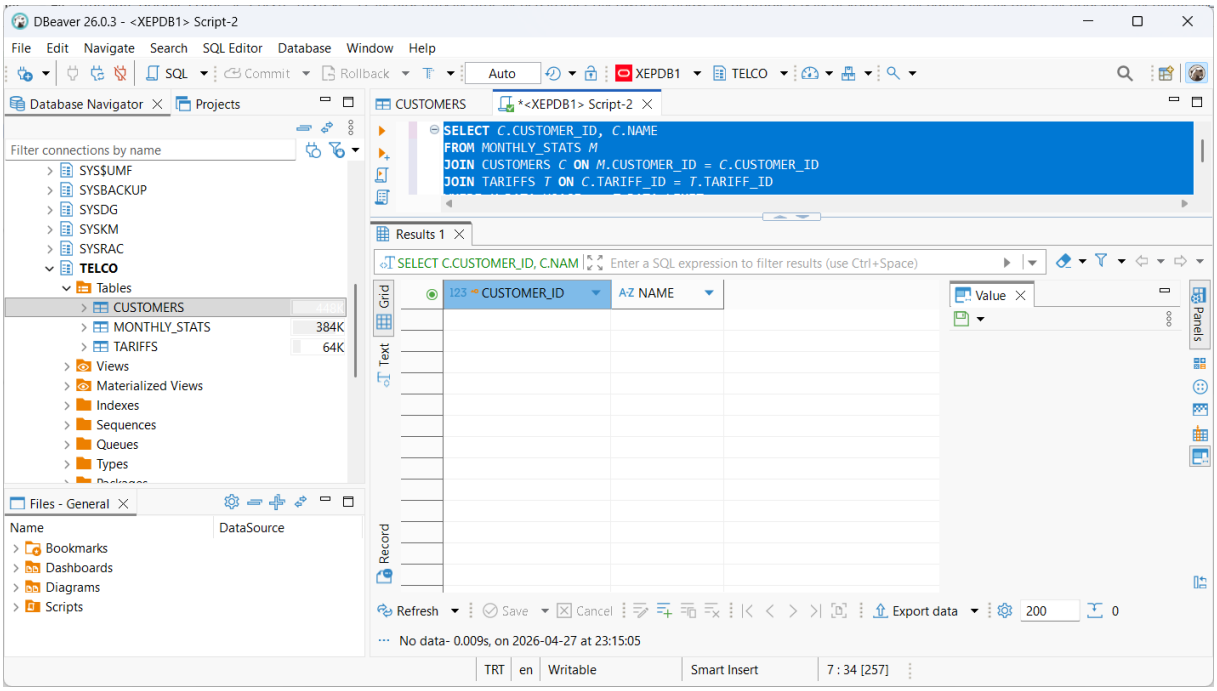
5.2 Identify the customers who have completely exhausted all of their package limits (data, minutes, and SMS).

We join the tables to get a full view of their limits and their actual usage side by side. We then ensure that their data, minutes, and SMS usage all individually meet or exceed the allowed amounts, though it's totally normal if nobody in the dataset actually hits all three limits simultaneously.

Query:

```
SELECT C.CUSTOMER_ID, C.NAME
FROM MONTHLY_STATS M
JOIN CUSTOMERS C ON M.CUSTOMER_ID = C.CUSTOMER_ID
JOIN TARIFFS T ON C.TARIFF_ID = T.TARIFF_ID
WHERE M.DATA_USAGE >= T.DATA_LIMIT
  AND M.MINUTE_USAGE >= T.MINUTE_LIMIT
  AND M.SMS_USAGE >= T.SMS_LIMIT;
```

Output:



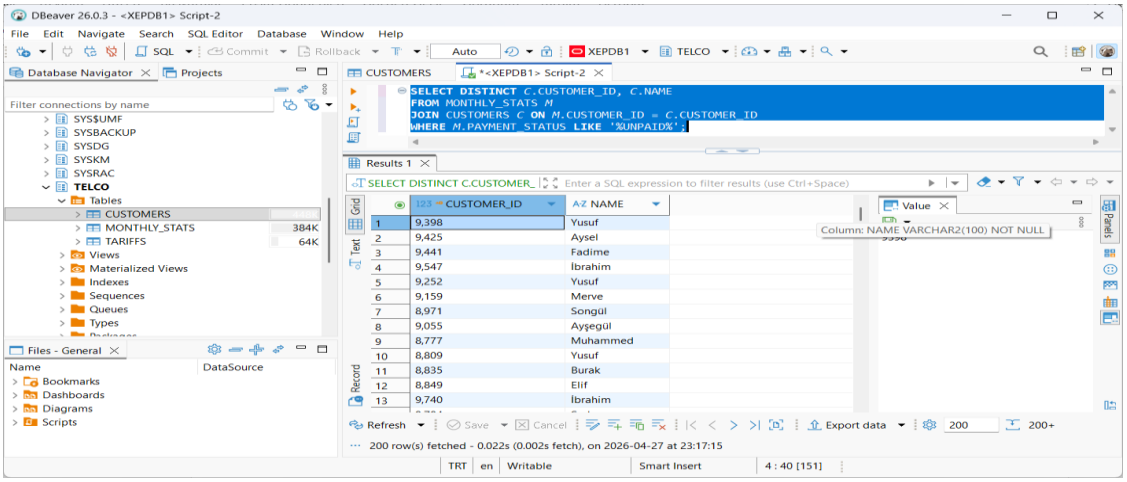
6.1 Find the customers who have unpaid fees.

We are looking for any usage record that has been marked as unpaid. After finding these records, we link back to the customers table to grab their actual names instead of just showing numbers. Using DISTINCT ensures that even if a customer missed multiple payments, their name only shows up once on our list.

Query:

```
SELECT DISTINCT C.CUSTOMER_ID, C.NAME
FROM MONTHLY_STATS M
JOIN CUSTOMERS C ON M.CUSTOMER_ID = C.CUSTOMER_ID
WHERE M.PAYMENT_STATUS LIKE '%UNPAID%';
```

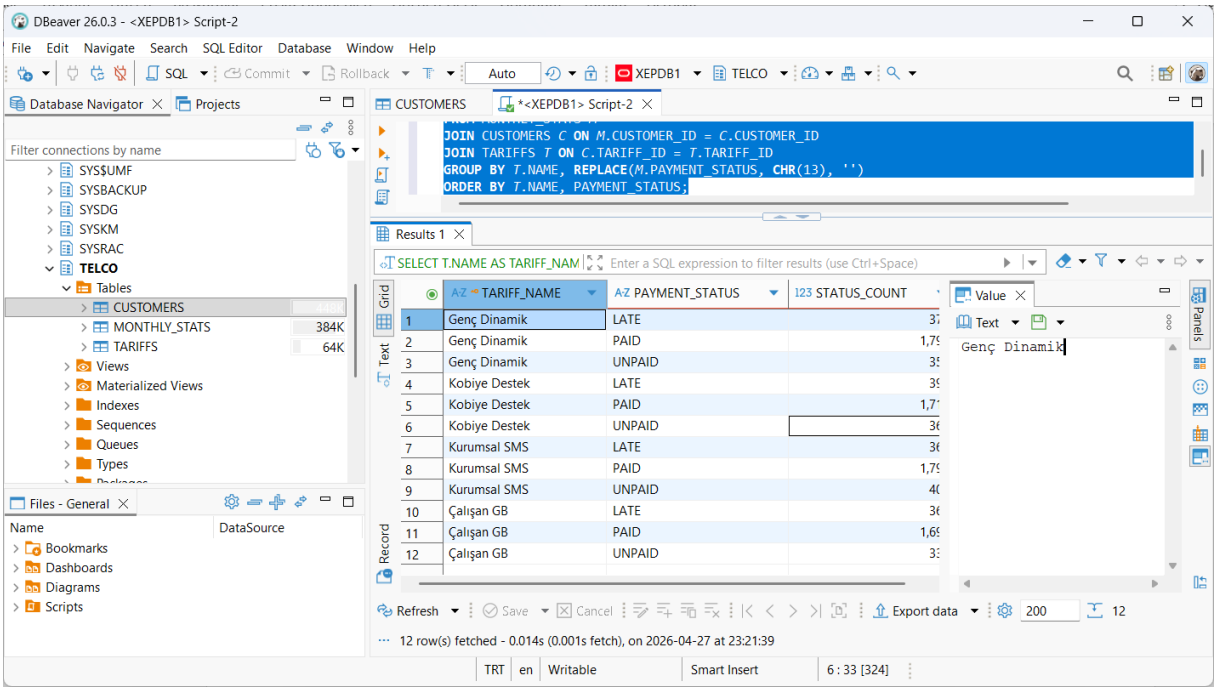
Output:



6.2 Find the distribution of all payment statuses across the different tariffs.

To understand how payment statuses are distributed across different tariffs, we first combine the MONTHLY_STATS, CUSTOMERS, and TARIFFS tables by matching their related IDs. After linking this data together, we group the results by each tariff name and the cleaned payment status (with any unwanted line break characters removed). This allows us to count how many records fall into each status category within every tariff. Finally, we sort the output by tariff name and payment status so the results are easier to scan and interpret.

Query:

```
SELECT T.NAME AS TARIFF_NAME, REPLACE(M.PAYMENT_STATUS, CHR(13), '') AS
PAYMENT_STATUS, COUNT(M.ID) AS STATUS_COUNT
FROM MONTHLY_STATS M
JOIN CUSTOMERS C ON M.CUSTOMER_ID = C.CUSTOMER_ID
JOIN TARIFFS T ON C.TARIFF_ID = T.TARIFF_ID
GROUP BY T.NAME, REPLACE(M.PAYMENT_STATUS, CHR(13), '')
ORDER BY T.NAME, PAYMENT_STATUS;
```

Output: